

PRIYANSHU SHARMA (23/38007)

1. **Write a program to perform encryption and decryption using Caesar cipher (substitutional cipher).**

```
def encrypt(text, shift):
    result = ""

    for char in text:
        if char.isupper():
            result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
        elif char.islower():
            result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
        else:
            result += char

    return result

def decrypt(text, shift):
    return encrypt(text, 26 - shift)

text = "Hello World"
shift = 3

print(f"Original text: {text}")

encrypted = encrypt(text, shift)
print(f"Encrypted text: {encrypted}")

decrypted = decrypt(encrypted, shift)
print(f"Decrypted text: {decrypted}")
```

OUTPUT:

```
Original text: Hello World
Encrypted text: Koor Zruog
Decrypted text: Hello World
```

2. **Write a program to perform encryption and decryption using Rail Fence Cipher (transpositional cipher).**

```
def encrypt(text, rails):
    if rails == 1:
        return text

    fence = [''] * rails
    rail = 0
    direction = 1

    for char in text:
        fence[rail] += char
        rail += direction

        if rail == 0 or rail == rails - 1:
            direction = -direction

    return ''.join(fence)

def decrypt(text, rails):
    if rails == 1:
        return text

    fence = [0] * rails
    rail = 0
    direction = 1

    for i in range(len(text)):
        fence[rail] += 1
        rail += direction

        if rail == 0 or rail == rails - 1:
            direction = -direction

    rows = []
    index = 0
    for count in fence:
        rows.append(text[index:index + count])
        index += count

    result = ''
    rail = 0
    direction = 1
    pos = [0] * rails

    for i in range(len(text)):
        result += rows[rail][pos[rail]]
        pos[rail] += 1
        rail += direction

        if rail == 0 or rail == rails - 1:
            direction = -direction
```

```
        return result

text = "HELLO WORLD"
rails = 3

print(f"Original text: {text}")

encrypted = encrypt(text, rails)
print(f"Encrypted text: {encrypted}")

decrypted = decrypt(encrypted, rails)
print(f"Decrypted text: {decrypted}")
```

OUTPUT:

```
Original text: HELLO WORLD
Encrypted text: HOREL OLLWD
Decrypted text: HELLO WORLD
```

3. **Write a Python program that defines a function and takes a password string as input and returns its SHA-256 hashed representation as a hexadecimal string.**

```
import hashlib

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

password = input("Enter password: ")
hashed = hash_password(password)
print(f"SHA-256 Hash: {hashed}")
```

OUTPUT:

```
Enter password: priyanshusharmaisthebest1474
SHA-256 Hash: 4e54f302274e383cf4d2689a6371ff1160af13c5b7c97102ebdaa579ad86e9ff
```

4. Write a Python program that reads a file containing a list of usernames and passwords, one pair per line (separated by a comma). It checks each password to see if it has been leaked in a data breach. You can use the "Have I Been Pwned" API (<https://haveibeenpwned.com/API/v3>) to check if a password has been leaked.

```
import hashlib
import requests

def check_password_leaked(password):
    sha1_hash = hashlib.sha1(password.encode()).hexdigest().upper()
    prefix = sha1_hash[:5]
    suffix = sha1_hash[5:]

    url = f"https://api.pwnedpasswords.com/range/{prefix}"
    response = requests.get(url)

    if response.status_code != 200:
        return None

    hashes = response.text.splitlines()
    for line in hashes:
        hash_suffix, count = line.split(':')
        if hash_suffix == suffix:
            return int(count)

    return 0

def check_file(filename):
    try:
        with open(filename, 'r') as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue

                parts = line.split(',')
                if len(parts) != 2:
                    print(f"Invalid format: {line}")
                    continue

                username, password = parts[0].strip(), parts[1].strip()
                count = check_password_leaked(password)

                if count is None:
                    print(f"{username}: Error checking password")
                elif count > 0:
                    print(f"{username}: LEAKED ({count} times)")
                else:
                    print(f"{username}: Safe")

    except FileNotFoundError:
        print(f"File '{filename}' not found")
```

```
filename = input("Enter filename (e.g., passwords.txt): ")  
check_file("passwords.txt")
```

OUTPUT:

```
Enter filename (e.g., passwords.txt): passwords.txt  
john: LEAKED (2031380 times)  
alice: LEAKED (21969901 times)  
bob: LEAKED (1138064 times)  
sarah: LEAKED (116434 times)  
mike: LEAKED (180041686 times)  
emma: LEAKED (5402944 times)  
david: LEAKED (41213657 times)  
lisa: LEAKED (669691 times)
```

5. **Write a Python program that generates a password using a random combination of words from a dictionary file.**

```
import random

def generate_password(dict_file, num_words=4):
    try:
        with open(dict_file, 'r') as f:
            words = [line.strip() for line in f if line.strip()]

        if len(words) < num_words:
            print("Not enough words in dictionary")
            return None

        selected = random.sample(words, num_words)
        return '-'.join(selected)

    except FileNotFoundError:
        print(f"Dictionary file '{dict_file}' not found")
        return None

dict_file = input("Enter dictionary file (e.g., words.txt): ")
num_words = int(input("Enter number of words (default 4): ") or "4")

password = generate_password(dict_file, num_words)
if password:
    print(f"Generated password: {password}")
```

OUTPUT:

```
Enter dictionary file (e.g., words.txt): dictionary.txt
Enter number of words (default 4): 4
Generated password: silver-cascade-diamond-dragon
```

6. **Write a Python program that simulates a brute-force attack on a password by trying out all possible character combinations.**

```
import itertools
import string
import time

def brute_force(target_password, charset, max_length):
    attempts = 0
    start_time = time.time()

    for length in range(1, max_length + 1):
        for attempt in itertools.product(charset, repeat=length):
            attempts += 1
            guess = ''.join(attempt)

            if attempts % 10000 == 0:
                print(f"Attempts: {attempts}, Current: {guess}")

            if guess == target_password:
                elapsed = time.time() - start_time
                print(f"\nPassword found: {guess}")
                print(f"Attempts: {attempts}")
                print(f"Time: {elapsed:.2f} seconds")
                return True

    print(f"\nPassword not found after {attempts} attempts")
    return False

target = input("Enter target password: ")
max_len = int(input("Enter max length to try: "))

print("\nCharset options:")
print("1. Digits only (0-9)")
print("2. Lowercase letters (a-z)")
print("3. Alphanumeric (a-z, 0-9)")
print("4. All printable characters")

choice = input("Select charset (1-4): ")

if choice == '1':
    charset = string.digits
elif choice == '2':
    charset = string.ascii_lowercase
elif choice == '3':
    charset = string.ascii_lowercase + string.digits
else:
    charset = string.ascii_letters + string.digits + string.punctuation

print(f"\nStarting brute force attack...")
print(f"Charset: {charset}")
print(f"Max length: {max_len}\n")
```

```
brute_force(target, charset, max_len)
```

OUTPUT 1:

```
Enter target password: 12345
Enter max length to try: 5

Charset options:
1. Digits only (0-9)
2. Lowercase letters (a-z)
3. Alphanumeric (a-z, 0-9)
4. All printable characters
Select charset (1-4): 1

Starting brute force attack...
Charset: 0123456789
Max length: 5

Attempts: 10000, Current: 8889
Attempts: 20000, Current: 08889

Password found: 12345
Attempts: 23456
Time: 4.72 milliseconds
```

OUTPUT 2:

```
Enter target password: cat
Enter max length to try: 3

Charset options:
1. Digits only (0-9)
2. Lowercase letters (a-z)
3. Alphanumeric (a-z, 0-9)
4. All printable characters
Select charset (1-4): 2

Starting brute force attack...
Charset: abcdefghijklmnopqrstuvwxyz
Max length: 3

Password found: cat
Attempts: 2074
Time: 0.00 milliseconds
```


OUTPUT 3:

```
Enter target password: ab12
Enter max length to try: 4

Charset options:
1. Digits only (0-9)
2. Lowercase letters (a-z)
3. Alphanumeric (a-z, 0-9)
4. All printable characters
Select charset (1-4): 3

Starting brute force attack...
Charset: abcdefghijklmnopqrstuvwxyz0123456789
Max length: 4

Attempts: 10000, Current: gy1
Attempts: 20000, Current: oot
Attempts: 30000, Current: wel
Attempts: 40000, Current: 34d
Attempts: 50000, Current: abt5

Password found: ab12
Attempts: 50285
Time: 12.00 milliseconds
```

OUTPUT 4:

```
Enter target password: a#
Enter max length to try: 2

Charset options:
1. Digits only (0-9)
2. Lowercase letters (a-z)
3. Alphanumeric (a-z, 0-9)
4. All printable characters
Select charset (1-4): 4

Starting brute force attack...
Charset: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
Max length: 2

Password found: a#
Attempts: 159
Time: 0.00 milliseconds
```